2024 18th International Conference on
Control, Automation, Robotics and Vision (ICARCV)
December 12-15, 2024. Dubai

Prioritized Planning For Large-Scale Multiple-AGV Scheduling Problem In Smart Manufacturing

Yao Guo, Jiarong Yao, Jiangpeng Li, Rong Su, and KV Ling

2024 18th International Conference on Control, Automation, Robotics and Vision (ICARCV) | 979-8-3315-1849-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICARCV63323.2024.10821683

***Abstract*— Robotics and automation is one of crucial trend in smart manufacturing to improve production efficiency. Automated guided vehicles (AGVs) are a type of mobile robot used for material handling and have become widely utilized to achieve transportation automation. The usage of multiple AGVs introduces potential risks, such as traffic conflicts and safety risk. To meet high production demands, numerous shop floors are set up for large-scale manufacturing. Thus, reasonable and efficient AGV scheduling is vital for real-world operations.**

This paper proposes an efficient and scalable prioritized planning algorithm for large-scale multiple-AGV scheduling problem in manufacturing. The algorithm sequentially addresses two primary sub-problems: job assignment and conflict-free routing. The results of job assignment dictate the routes taken by the AGVs. In job assignment, jobs are allocated sequentially based on their pickup times. In conflict-free routing, AGV priorities are predefined, ensuring that higher priority AGVs maintain their movement while adjustments are made only to lower priority AGV plans when conflicts arise. Simulation is conducted on two real shop floor layouts and demonstrates the effectiveness and high efficiency of proposed algorithm. Even in a large-scale layout with 500 jobs and 20 AGVs, the computation time is only around 21 seconds.

I. INTRODUCTION

The pursuit of smart manufacturing, epitomized by Industry 4.0, has brought robotics and automation to the forefront of the manufacturing industry. Automated Guided Vehicles (AGVs) have increasingly been adopted to achieve automated material handling in factories. The deployment of multiple AGVs introduces the challenge of reasonable scheduling and coordination. Irrational plan can lead to resource wastage, production delays, and even collisions [1]. In practical manufacturing settings, the presence of large factories, numerous AGVs, and various transport jobs makes large-scale AGV scheduling a significant concern. This scheduling problem inherently involves high computational complexity and solution efficiency is crucial in real-world applications [2].

The AGV scheduling problem has received significant attention in research. Traditional approaches, such as exact algorithms [3]–[5], provide optimal solutions but are highly constrained by problem scale. Consequently, heuristic approaches have become essential, offering practical solutions although they do not guarantee optimality. Numerous studies in the literature have explored heuristic approaches for AGV scheduling in manufacturing [6]–[8]. The AGV scheduling problem in manufacturing comprises two primary sub-problems: job assignment and conflict-free routing. The literature mentioned above adopt coupled approach, where both sub-problems are tackled simultaneously.

Another approach of interest in this paper is the decoupled approach, where two sub-problems are separated and solved sequentially. The literature [9]–[11] demonstrate efficiency, especially in large-scale AGV scheduling scenarios. Prioritized planning involves organizing job/robot based on their relative importance or urgency, where plan job/robot sequentially. Prioritized planning has been shown effective in job assignment [12], [13] and in multiple robot path planning and coordination [14]–[16].

This paper proposes an efficient prioritized planning algorithm for large-scale multiple-AGV scheduling problems in manufacturing. AGV scheduling problems are addressed by decomposing it into two sub-problems, solving job assignment problem to provide inputs for conflict-free routing problem. In job assignment, jobs are prioritized based on their pickup time, and then assigned one by one considering two objectives, makespan and earliness-tardiness. In conflict-free routing, AGVs are organized sequentially based on their predefined priority and only the movements of lower priority AGV are allowed to be adjusted to avoid conflicts. The movement of AGVs is represented by occupy time windows for nodes and edges and node/edge conflict is visualized by the overlap between time windows for one node/ bidirectional track. Conflicts are eliminated by adjusting the time windows of lower priority AGV to resolve overlap.

The remaining structure of this paper is as follows: Section II presents the problem statement, including basic notations and assumptions. Section III outlines the algorithm steps and provides analysis. Section IV gives simulation setting and results for two real shop floor layouts. Finally, Section V concludes the paper and discusses future work.

II. PROBLEM STATEMENT

Job assignment involves the assignment of jobs to AGVs. Let job set is denoted by J with index j and AGV set is denoted by V with index i. Each transport job j includes pickup station, delivery station, pickup time, and delivery time. The pickup station is where materials are collected, while the delivery station is the destination where materials are transported to. The pickup and delivery times is the time requirement for material handling. In addition, we consider that AGVs require a certain amount time to operate pickup or delivery materials at stations.

The pickup and delivery requirements of job j and the time required for pickup or delivery operations for AGVs are denoted below.

Yao Guo, Jiarong Yao, Jiangpeng Li, Rong Su, and KV Ling are with the School of Electrical and Electronic Engineering, Nanyang Technological University, 639798, Singapore.

Corresponding author: KV Ling. Email: ekvling@ntu.edu.sg

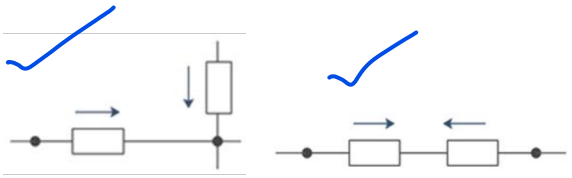
Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 27,2026 at 08:29:44 UTC from IEEE Xplore. Restrictions apply.

- p_j: the pickup station for job j.
- t_j: the desired pickup time for job j.
- p'_j: the delivery station for job j.
- t'_j: the desired delivery time for job j.
- η: the loading time for AGVs to pickup materials.
- η': the unloading time for AGVs to delivery materials.

Assumptions in job assignment:

1) One job is assigned to one AGV to handle. No merging of multiple jobs on one AGV, or splitting of one job to multiple AGVs exists.
2) One AGV is capable of handling at most one job at a given time.

The conflict-free routing problem entails finding a route for one AGV from an origin to a destination without intersecting with routes of other AGVs. Two primary types of conflicts are identified: node conflicts occur when paths of multiple AGVs intersect at nodes, while edge conflicts arise when AGVs collide head-on along a bidirectional track. Two conflicts are demonstrated in Figure 1 of [11].



(a) Node conflict (b) Edge conflict

Fig. 1. Demonstration of two types of conflicts

The layout is denoted by $G = \{N, E\}$, where N is the set of nodes indexed by n and E is the set of edges indexed by e. Assume there is one parking station in layout, which can accommodate multiple AGVs parking simultaneously. The safety separation time is defined here, which is the minimum time interval required to maintain safe distances between AGVs.

- π: the parking station in layout
- δ : the minimum safety separation time between AGVs.

Assumptions in conflict-free routing are given below.

1) AGVs move at a constant speed.
2) AGVs start from the parking station and end at the parking station.
3) Each node except the parking station can be visited by at most one AGV at a given time.
4) Each bidirectional track can be occupied by at most one AGV at a given time.
5) AGVs can pause at nodes but are not permitted to pause at edges.
6) All AGVs are available at starting time 0.

III. Prioritized Planning

A. Job assignment

In job assignment problem, we omit the conflict between AGVs. The Dijkstra algorithm is used to find the shortest path between any two nodes in shop floor. The length of the shortest path is divided by the AGV speed to determine the non-stop travel time between two nodes. We give notations to denote them below.

- $r_{nn'}$: the shortest path between two nodes $n, n' \in N$.
- $\delta_{nn'}$: the non-stop travel time of the shortest path between two nodes $n, n' \in N$.

where $r_{nn'}$ denotes the set of nodes with sequence. For example, $r_{n,n'} = \{n, n'', n'\}$ indicates the shortest path from node n to n' starts at n, passes through n'' and ends at n'.

The process of assigning job j to AGV i with known time and position information vt_i, s_i is depicted in Figure 2. AGV i travels from current position to the job pickup station and taking loading time operate pickup, then travels to job delivery station and spend unloading time to delivery materials. The route is $s_i \rightarrow p_j \rightarrow p'_j$. We directly use the shortest path obtained by Dijkstra algorithm and the route can be denoted as $r_{s_i p_j} \cup r_{p_j p'_j}$.

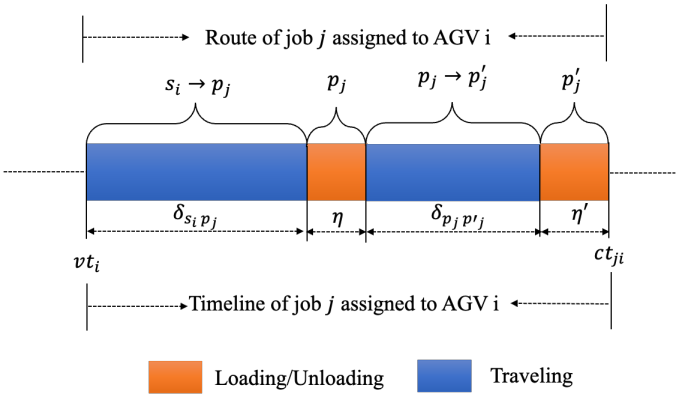


Fig. 2. Demonstration of route and timeline when assign job j to AGV i

A fitness value function is provided to evaluate the performance of assigning job j to AGV i with known time and position information vt_i, s_i. Two objectives are considered in the fitness value computation: job completion time and earliness-tardiness. The fitness value function is

$$F_{ji}(vt_i, s_i) = ct_{ji} + \lambda(et_{ji} + dt_{ji}) \tag{1}$$

where λ is a weight parameter, ct_{ji} is the job completion time, et_{ji} is the earliness for pickup operation, dt_{ji} is the tardiness for delivery operation and can be denoted as

$$\begin{cases} ct_{ji} = vt_i + \delta_{s_i p_j} + \eta + \delta_{p_j p'_j} + \eta' \\ et_{ji} = max(0, t_j - (vt_i + \delta_{s_i p_j} + \eta)) \\ dt_{ji} = max(0, ct_{ji} - t'_j) \end{cases} \tag{2}$$

Based on the desired pickup time, we establish job priorities where jobs with earlier pickup times receive higher priority. For the current highest priority job, We assign it to the AGV with minimum fitness value and update AGV time and position information accordingly. The prioritized planning algorithm for job assignment is given in Algorithm 1. After assigning jobs, an additional sub-route is added for each AGV to return to the parking station. The result of the job assignment is the set for AGVs' route, denoted as $\mathcal{R} = \{R_i \mid i \in V\}$, which serves as the input for the conflict-free routing problem.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 27,2026 at 08:29:44 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Prioritized Planning for Job Assignment

Require: Job set J, AGV set V
Ensure: Routes of all AGVs $\mathcal{R} = \{R_i \mid i \in V\}$

Initialize $R_i \leftarrow \emptyset$, $vt_i \leftarrow 0$, and $s_i \leftarrow \pi$ for each AGV $i \in V$
while $J \neq \emptyset$ **do**
  Select job j with earliest pickup time from J
  for each AGV $i \in V$ **do**
    Calculate fitness value $F_{ji}(vt_i, s_i)$
  end for
  Assign job j to AGV i^* with minimum F_{ji^*}
  Update AGV route $R_{i^*} \leftarrow R_{i^*} \cup r_{s_{i^*}p_j} \cup r_{p_j p'_j}$
  Update $vt_{i^*} \leftarrow ct_{ji^*}$ and $s_{i^*} \leftarrow p'_j$
  Remove job j from J
end while
for each AGV $i \in V$ **do**
  Update $R_i \leftarrow R_i \cup r_{s_i\pi}$
end for

B. Conflict-free routing

We have obtained the route of AGVs in job assignment and the route of AGV i is the set of nodes with sequence, denoted as $R_i = \{n_1, n_2, ...\}$. We clarify the subscript of notation n only denotes the sequence, rather than node name in layout. For example, n_1 is the first node in the R_i. Let $P_i = \{[lt_1, rt_1], [lt_2, rt_2], ...\}$ denotes the plan of AGV i with the occupy time window of each node in R_i. For edge $e(n_1, n_2)$, the occupy time window is $[rt_1, lt_2]$. The set of AGVs' plan is $\mathcal{P} = \{P_i \mid i \in V\}$.

In the conflict-free routing problem, the priority for AGVs is predefined and fixed. We arrange the AGV set V in descending order of priority, denoted as $V = \{V_1, V_2, \ldots\}$, where V_1 is the AGV with the highest priority. We determine the AGV plan sequentially according to descending priority. Movement adjustments are only allowed for AGVs with lower priority when there is a conflict. For V_1, non-stop travel except for pickup and delivery operations directly denotes its plan. V_2 considers the plan of V_1, while V_3 considers the plan of both V_1 and V_2, and so forth. When considering the plan of higher priority AGVs, their movements are determined and fixed, which are rearranged as the set of time windows for each node and edge denoted by $\mathcal{W} = \{W_n | n \in N\}$ and $\mathcal{W}' = \{W'_e | e \in E\}$. Planning AGV i is to add $|R_i|$ time windows into P_i and added time windows has no conflict with $\mathcal{W}, \mathcal{W}'$. The prioritized planning for conflict-free routing is given in Algorithm 2.

Next, we discuss four conflict cases and their criteria. Conflict arise when there is an overlap between time windows for one node or two edges composing one bidirectional track. The visualization of overlap cases between time windows are provided in Figure 3. The newly added time window for node n or edge $e(n, n')$ is depicted by a blue rectangle with boundaries $[lt, rt]$. The determined time windows in $W_n/W'_{e'(n',n)}$ are represented by white rectangles, where both sides are denoted by a notation with a hat, indicating

Algorithm 2 Prioritized Planning for Conflict-free Routing

Require: AGV set V, Routes of all AGVs $\mathcal{R} = \{R_i \mid i \in V\}$
Ensure: Plan of all AGVs $\mathcal{P} = \{P_i \mid i \in V\}$

Initialize $W_n \leftarrow \emptyset$ for each node $n \in N$
Initialize $W'_e \leftarrow \emptyset$ for each edge $e \in E$
while $V \neq \emptyset$ **do**
  Select AGV i with highest priority in V
  Initialize time $lt_1 \leftarrow 0$ for AGV i
  for Node sequence index m from 1 to $|R_i|$ **do**
    if Node n_{m-1} exists in R_i **then**
      Determine time window of edge $e(n_{m-1}, n_m)$ as $[rt_{m-1}, rt_{m-1} + \delta_{n_{m-1},n_m}]$
      if Conflict occurs at edge $e(n_{m-1}, n_m)$ **then**
        Find conflict-free plan P_i based on Algorithm 3
      end if
      $lt_m \leftarrow rt_{m-1} + \delta_{n_{m-1}n_m}$
    end if
    $rt_m \leftarrow lt_m$
    if Node n_m is the L/U station **then**
      $rt_m \leftarrow lt_m + \eta/\eta'$
    end if
    $P_i = P_i \cup [lt_m, rt_m]$
    Determine time window of node n_m as $[lt_m, rt_m]$
    if Conflict occurs at node n_m **then**
      Find conflict-free plan P_i based on Algorithm 3
    end if
  end for
  Update $\mathcal{W}$ and $\mathcal{W}'$ based on P_i
  Remove AGV i from V
end while

boundaries are fixed. The criteria for four conflict cases are detailed in Table I.

TABLE I
CRITERIA FOR FOUR CONFLICT CASES

Conflict case	Criteria
Case 1	$\overline{lt} - \delta < rt < \overline{rt} + \delta$
Case 2	$\overline{lt} - \delta < lt < \overline{rt} + \delta$
Case 3	$(lt \leq \overline{lt} - \delta) \cap (\overline{rt} + \delta \leq rt)$
Case 4	$(\overline{lt} - \delta \leq lt) \cap (rt \leq \overline{rt} + \delta)$ $\cap (\overline{lt} - \delta < rt) \cap (lt < \overline{rt} + \delta)$

The time windows in higher priority AGVs' plan is rearranged to form $W_n/W'_{e'(n',n)}$, indicating white rectangle are fixed and only blue rectangle are allowed to be adjusted to prevent overlap. We adopt pause action for AGVs to avoid conflict and the blue time window can be slid to right. The sliding time range is introduced in Figure 4. In Figure 4(a), blue time window $[lt, rt]$ has an overlap with white time window $[\overline{lt}_1, \overline{rt}_1]$. We aims to find an enough space between two adjacent white time windows to place the blue time window. The space between $[\overline{lt}_1, \overline{rt}_1]$ and $[\overline{lt}_2, \overline{rt}_2]$ are too narrow. There is an enough space between $[\overline{lt}_2, \overline{rt}_2]$ and

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 27,2026 at 08:29:44 UTC from IEEE Xplore. Restrictions apply.

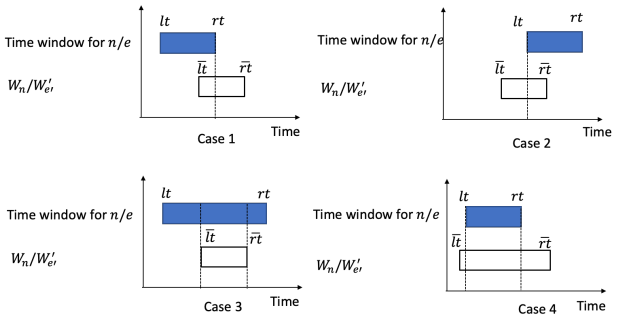
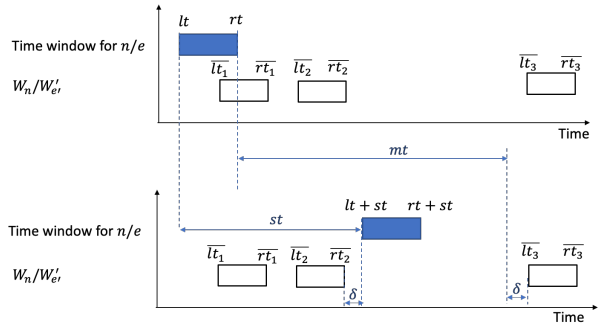
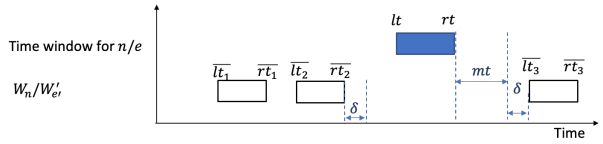


Fig. 3. Four conflict cases

$[\overline{lt}_3, \overline{rt}_3]$, i.e., $\overline{lt}_3 - \overline{rt}_2 \geq rt - lt + 2\delta$, so that we can slide the blue time window to the space between these two white time windows. The minimum sliding time $st = \overline{rt}_2 + \delta - lt$, meanwhile there is a maximum sliding time $mt = \overline{lt}_3 - \delta - rt$, which denotes that if blue time window $[lt, rt]$ slides to right more than mt and will have conflict with the next white time window $[\overline{lt}_3, \overline{rt}_3]$.



(a) Sliding time range in conflict case



(b) Sliding time range in conflict-free case

Fig. 4. Demonstration of sliding time range

We define a function to denote the progress of finding the minimum and maximum sliding time required to slide one time window to resolve overlap below.

$$st, mt = f(lt, rt, W_n/W'_{e'}) \tag{3}$$

If there is no conflict as Figure 4(b), the function will return to $st = 0$ and $mt = \overline{lt}_3 - \delta - rt$, indicating the blue window do not need slide but also has an allowable maximum sliding time to right to keep no conflict.

The slide of one time window may cause the discontinuity in AGV plan. For example, we slide one node time window in plan and $P_i = \{[lt_1, rt_1], [lt_2, rt_2], [lt_3 + st, rt_3 + st], ...\}$. The occupy time window for edge $e(n_2, n_3)$ is $[rt_2, lt_3 + rt]$, which exceeds the edge travel time $(lt_3 - rt_2)$ and indicates that AGV pause at the edge. This violates our assumption in conflict-freer routing.

Therefore, slide one time window requires to slide the previous time windows in P_i accordingly until the parking station. If there is a conflict for previous time windows after slide, the new required sliding time should be cumulative and is the extend length of the previous nearest parking station, indicating the AGV wait at the parking station for the total sum of required sliding time. Time windows for other nodes except the parking station and edges are slid with unchanged time length. Eliminating conflict by adjusting time windows in the plan of lower priority AGV is given in Algorithm 3.

Algorithm 3 Find a conflict-free plan to eliminate conflict at n_p

Require: Route $R_i = \{n_1, n_2, ...\}$
Plan $P_i = \{[lt_1, rt_1], [lt_2, rt_2], ..., [lt_p, rt_p]\}$
Determined time window set $\mathcal{W} = \{W_n | n \in N\}$ and $\mathcal{W}' = \{W'_e | e \in E\}$
Maximum iteration times k_{max}
Ensure: Conflict-free Plan P_i

Initialize $st^* \leftarrow 0$
for Iteration index k from 1 to k_{max} **do**
  Initialize $wt^* \leftarrow \infty$
  for Node sequence index m from p to 2 **do**
    The time window of node n_m after sliding right by st^* is denoted by $[lt_m + st^*, rt_m + st^*]$
    $st, mt = f(lt_m + st^*, rt_m + st^*, W_{n_m})$
    $mt^* = min(st^* + mt, mt^*), st^* = st^* + st$
    if $st^* > mt^*$ **then**
      break
    end if
    The time window of edge $e(n_{m-1}, n_m)$ after sliding right by st^* is denoted by $[rt_{m-1} + st^*, lt_m + st^*]$.
    $st, mt = f(rt_{m-1} + st^*, lt_m + st^*, W'_{e(n_m, n_{m-1})})$
    $mt^* = min(st^* + mt, mt^*), st^* = st^* + st$
    if $st^* > mt^*$ **then**
      break
    end if
    if node n_{m-1} is the parking station **then**
      break
    end if
  end for
  if node n_{m-1} is the parking station **then**
    break
  end if
end for
for Node sequence index m from p to 2 **do**
  Slide time window of node n in P_i to right by st^*
  if node n_{m-1} is the parking station **then**
    Extend time window of node n in P_i right by st^*
    break
  end if
end for

In addition, we give an illustrative example with numerical

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 27,2026 at 08:29:44 UTC from IEEE Xplore. Restrictions apply.

data to improve the readability of Algorithm 3. Assume $R_i = \{n_1, n_2, ...\}$ and $P_i = \{[0, 6], [10, 14]\}$. n_1 is the parking station and safety time is $\delta = 2$ The time windows for refereed nodes and edges in W, W' are

$$\begin{aligned} W_{n_2} &= \{[8, 12], [22, 26][50, 54]\} \\ W'_{e(n_2,n_1)} &= \{[10, 14], [30, 34]\} \end{aligned} \tag{4}$$

The time window of n_2, $[10, 14]$ has conflict with one window $[8, 12]$ in W_{n_2}, which is conflict case 2 in Figure 3. Based on Algorithm 3, we give the detailed steps below.

1) Initialize $st^* = 0$.
2) Initialize iteration time $k = 1$ and $mt^* = \infty$.
3) For node n_2, the time window is $[10 + 0, 14 + 0]$.
4) Run function $st, mt = f(10, 14, W_{n_2})$ and get $st = 4, mt = 6$.
5) $mt^* = min(0 + 6, \infty) = 6, st^* = 0 + 4 = 4$.
6) For edge $e(n_1, n_2)$, the time window is $[6+4, 10+4]$.
7) Run function $st, mt = f(10, 14, W'_{e(n_2,n_1)})$ and get $st = 6, mt = 14$.
8) $mt^* = min(4 + 14, 6) = 6, st^* = 4 + 6 = 10$.
9) Turn to next iteration $k = 2$ since $st^* > mt^*$.
10) Initialize $mt^* = \infty$.
11) For node n_2, the time window is $[10 + 10, 14 + 10]$.
12) Run function $st, mt = f(20, 24, W_{n_2})$ and get $st = 8, mt = 24$.
13) $mt^* = min(10 + 24, \infty) = 34, st^* = 10 + 8 = 18$.
14) For edge $e(n_1, n_2)$, the time window is [6+18, 10+18].
15) Run function $st, mt = f(24, 28, W'_{e(n_2,n_1)})$ and get $st = 0, mt = 0$.
16) $mt^* = min(18 + 0, 34) = 18, st^* = 18 + 0 = 18$.
17) For parking station n_1, stop iteration.
18) Slide time window of node n_2 to right by $st^* = 18$ as $[28, 32]$.
19) Extend time window of node n_1 to the right by st^* as $[0, 24]$.

The conflict-free plan is $P_i = \{[0, 24], [28, 32]\}$. New time window of edge $e(n_1, n_2)$ is $[24, 28]$ and it has no conflict with $W_{e(n_2,n_1)}$ and new time window of node n_2 is $[28, 32]$ that has no conflict with W_{n_2} either, indicating the conflict has been eliminated by Algorithm 3.

Algorithm 3 demonstrates the progress to eliminate node conflict and the adjusted time windows type sequence is node $\rightarrow$ edge $\rightarrow$ node ... $\rightarrow$ node. For edge conflict, the adjusted time window type sequence should be edge $\rightarrow$ node $\rightarrow$ edge ... $\rightarrow$ node. The algorithm to eliminate edge conflict can refer to Algorithm 3 with changed time window type sequence and the detail is not provided due to limited pages.

C. Algorithm analysis

In Algorithm 3, conflict is eliminated by adjusting time windows in the plan of lower priority AGV. If the iteration time k_{max} is huge enough, there is a guaranteed feasible conflict-free plan for all AGVs. Since the worst case is that one AGV star from the parking station after AGVs with higher priority completed all jobs and back to the parking station. The guaranteed feasible solution is crucial for practical manufacturing since it ensures reliable AGV scheduling

When there is a conflict, only adjustment for lower priority AGV is allowed. The AGV with higher priority has the shorter waiting time and usage time. With the increase in the number of AGVs, the usage time difference, i.e., load imbalance, between AGVs should be larger. In the following simulation section, load imbalance is evaluated.

IV. Simulation

The proposed algorithm are tested on two real shop floor layouts provided by our industry partner SCHAEFFLER. Various examples with different number of jobs and AGVs, denoted by $|J|$ and $|V|$ respectively, are listed. All code are written in Python 3.9 and run on a 3.2-GHz 8-core Apple M1 processor with 16GB RAM. Apart from two objectives, makespan and earliness-tardiness, load imbalance is listed to evaluate the plan quality. Load imbalance is defined as the maximum usage time difference among AGVs. AGV settings are: $\eta = \eta' = 16$, and $\delta = 10$.

A. Real shop floor layout 1

A real shop floor layout is shown in Figure 5, where nodes are depicted as circles with their names written inside. Tracks are represented as line segments connecting nodes, with arrows indicating direction. The travel time for each track is denoted by a value written above it in seconds. The layout is one real production line, consisting of 12 nodes, 16 edges, 4 bidirectional tracks and one parking station located at $N5$. The numerical result of 14 examples are presented in Table II.

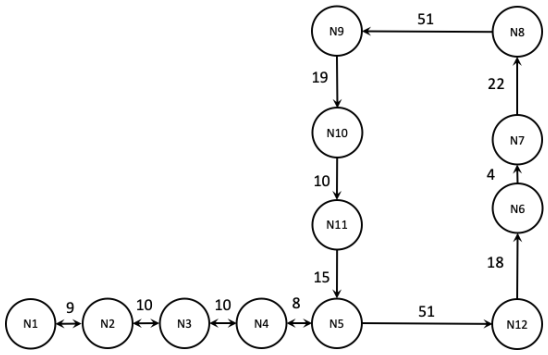


Fig. 5. Real shop floor layout 1

B. Real shop floor layout 2

A larger-scale shop floor layout is evaluated, which includes 44 nodes, 53 edges, 6 bidirectional tracks and one parking station. Due to the limitation of page number, the layout visualization is not provided. The numerical result is listed in Table III.

Examples 1-10, based on two layouts, show that increasing the number of jobs leads to a longer makespan. Examples 10-14 reveal that increasing the number of AGVs always reduces the makespan but increases load imbalance, consistent with algorithm analysis. For examples with layout 1, computation

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 27,2026 at 08:29:44 UTC from IEEE Xplore. Restrictions apply.

TABLE II

NUMERICAL RESULT FOR EXAMPLES BASED ON SHOP FLOOR LAYOUT 1

Ex	$\|J\|\&\|V\|$	Makespan(s)	Earliness-tardiness(s)	Load imbalance(s)	Computation time(s)
1	50&4	3254	0	362	0.09
2	100&4	6152	0	130	0.24
3	150&4	9220	0	326	0.51
4	200&4	12154	0	150	0.86
5	250&4	15222	0	346	1.34
6	300&4	18136	0	166	1.89
7	350&4	21204	0	362	2.72
8	400&4	24102	0	130	3.4
9	450&4	27170	0	326	4.17
10	500&4	30104	0	150	5.4
11	500&8	21308	0	6367	6.61
12	500&12	21396	0	11376	8.29
13	500&16	21100	0	13606	9.43
14	500&20	18894	0	12852	10.34

TABLE III

NUMERICAL RESULT FOR EXAMPLES BASED ON SHOP FLOOR LAYOUT 2

Ex	$\|J\|\&\|V\|$	Makespan(s)	Earliness-tardiness(s)	Load imbalance(s)	Computation time(s)
1	50&4	2718	0	336	0.19
2	100&4	5343	0	540	0.66
3	150&4	7735	0	717	1.46
4	200&4	10182	0	743	2.3
5	250&4	12696	0	1042	3.31
6	300&4	15169	0	1168	4.85
7	350&4	17434	0	1041	6.23
8	400&4	20077	0	1364	8.01
9	450&4	22703	0	1775	9.95
10	500&4	25198	0	1863	12.14
11	500&8	15910	0	4731	14.22
12	500&12	14197	0	6784	18.71
13	500&16	13418	0	7760	20.92
14	500&20	12966	0	8477	21.42

time remains in seconds. In example 14 with large-scale layout 2, 500 jobs and 20 AGVs, the computation time is just over 20 seconds, demonstrating the algorithm's efficiency.

V. CONCLUSION

This paper proposes an efficient prioritized planning algorithm for solving large-scale multiple-AGV scheduling problems in smart manufacturing, addressing both job assignment and conflict-free routing. The algorithm tackles these two sub-problems separately and sequentially. Prioritized planning is employed for sub-problems. AGV plan is represented by occupy time windows for nodes and edges, and conflict is eliminated by adjusting time windows in the plan of lower priority AGV to resolve overlap. The proposed algorithm reliably finds a feasible conflict-free plan, which is essential for practical manufacturing. In addition, simulation evaluates efficiency and scalability of the algorithm by two real shop floor layouts.

Given the widespread use of electric AGVs with charging requirements, integrating charging mechanisms into the algorithm is necessary. We identify current shortcomings in our proposed algorithm: specifically, only lower priority AGV can have its movements adjusted in case of conflicts, resulting in lengthy waiting times and significant load imbalances. Improving load balance is a key focus of future work.

ACKNOWLEDGEMENT

This research is supported by the Agency for Science, Technology and Research (A*STAR) under its IAF-ICP Programme ICP1900093 and the Schaeffler Hub for Advanced Research at NTU.

REFERENCES

[1] W. Lu, S. Guo, T. Song, and Y. Li, "Analysis of multi-agvs management system and key issues: A review," *Comput. Model. Eng. Sci*, vol. 131, no. 3, pp. 1197–1227, 2022.

[2] W.-Q. Zou, Q.-K. Pan, L. Wang, Z.-H. Miao, and C. Peng, "Efficient multiobjective optimization for an agv energy-efficient scheduling problem with release time," *Knowledge-Based Systems*, vol. 242, p. 108334, 2022.

[3] G. Desaulniers, A. Langevin, D. Riopel, and B. Villeneuve, "Dispatching and conflict-free routing of automated guided vehicles: An exact approach," *International Journal of Flexible Manufacturing Systems*, vol. 15, no. 4, pp. 309–331, 2003.

[4] S. B. Akkaya and M. A. Gökçe, "Intelligent scheduling and routing of a heterogenous fleet of automated guided vehicles (agvs) in a production environment with partial recharge," in *International Conference on Intelligent and Fuzzy Systems*. Springer, 2022, pp. 568–576.

[5] Y. Guo, J. Yao, R. Su, K. Ling, B. S. Han, and H. Y. A. Wong, "A milp model for conflict-free routing problem with partial charging in multiple-agv system," in *2024 IEEE 18th International Conference on Control & Automation (ICCA)*. IEEE, 2024, pp. 749–754.

[6] K. Nishida and T. Nishi, "Dynamic optimization of conflict-free routing of automated guided vehicles for just-in-time delivery," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 3, pp. 2099–2114, 2022.

[7] J. Li, Y. Guo, J. Yao, Z. Hu, R. Su, B. S. Han, and H. Y. A. Wong, "Hybrid metaheuristics for integrated dispatching and conflict-free vehicle routing of automated guided vehicles in flexible manufacturing systems," in *2023 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM)*. IEEE, 2023, pp. 134–139.

[8] X. Lyu, Y. Song, C. He, Q. Lei, and W. Guo, "Approach to integrated scheduling problems considering optimal number of automated guided vehicles and conflict-free routing in flexible manufacturing systems," *IEEE Access*, vol. 7, pp. 74 909–74 924, 2019.

[9] S. Riazi, K. Bengtsson, and B. Lennartson, "Energy optimization of large-scale agv systems," *IEEE Transactions on automation science and engineering*, vol. 18, no. 2, pp. 638–649, 2020.

[10] S. Riazi, T. Diding, P. Falkman, K. Bengtsson, and B. Lennartson, "Scheduling and routing of agvs for large-scale flexible manufacturing systems," in *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)*. IEEE, 2019, pp. 891–896.

[11] Y. Hu, L. Dong, and L. Xu, "Multi-agv dispatching and routing problem based on a three-stage decomposition method," *Mathematical Biosciences and Engineering*, vol. 17, no. 5, pp. 5150–5172, 2020.

[12] S. Woo, A. M. Keller, W. Choudhry, R. Trakhtengerts, S. Bakshi, Z. Yan, and D. Chen, "A development platform for real time routing of an autonomous ground vehicle to complete prioritized tasks," in *2019 American Control Conference (ACC)*. IEEE, 2019, pp. 1394–1399.

[13] S. MahmoudZadeh, D. Powers, K. Sammut, A. Lammas, and A. M. Yazdani, "Optimal route planning with prioritized task scheduling for auv missions," in *2015 IEEE International Symposium on Robotics and Intelligent Sensors (IRIS)*. IEEE, 2015, pp. 7–14.

[14] S. Ware and R. Su, "Incremental scheduling of discrete event systems," in *2016 13th International Workshop on Discrete Event Systems (WODES)*. IEEE, 2016, pp. 147–152.

[15] Y. Zhang, F. Wang, F. Fu, and Z. Su, "Multi-agv path planning for indoor factory by using prioritized planning and improved ant algorithm." *Journal of Engineering & Technological Sciences*, vol. 50, no. 4, 2018.

[16] R. Tai, J. Wang, and W. Chen, "A prioritized planning algorithm of trajectory coordination based on time windows for multiple agvs with delay disturbance," *Assembly Automation*, vol. 39, no. 5, pp. 753–768, 2019.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 27,2026 at 08:29:44 UTC from IEEE Xplore. Restrictions apply.